

YOU OS – Stoic Disciplined Coach

Full Technical Implementation Architecture Guide (Conversational V2.1)

1. Executive Overview

YouOS is a conversational-first, stateful discipline agent built using Laravel, Vue.js, MySQL, OpenRouter LLM, and Telegram Webhooks.

This document provides full implementation-level guidance covering backend domain architecture, conversational orchestration, LLM validation, reminders, Telegram integration, scheduling, security, scalability, and frontend integration.

The system must feel agentic, proactive, and disciplined, while remaining structurally strict internally.

2. System Architecture Layers

Frontend Layer (Vue.js SPA + Web Chat)

Transport Layer (Telegram Webhook + REST API)

Chat Orchestrator (Core Brain)

Intent Extraction Layer (LLM JSON-only)

Domain Services (Doctrine, Check-in, Decision, Reminder, Reflection)

Persistence Layer (MySQL)

External LLM Layer (OpenRouter)

3. Conversational Core Design

ChatOrchestrator is the central application service.

- Receive normalized message
- Load user state + doctrine summary
- Load recent conversation memory (last 10–15 messages)
- Call LLM intent extractor
- Validate strict JSON output
- Route to appropriate domain handler
- Store structured data (if confirmed)
- Return assistant response

4. Database Schema – Core Tables

conversations (id, user_id, channel, external_id, timestamps)

conversation_messages (id, conversation_id, role, content, structured_json, timestamps)

pending_actions (id, user_id, conversation_id, action_type, payload_json, expires_at)

reminders (id, user_id, channel, message, scheduled_for, status)

doctrines (one per user, structured JSON fields)

daily_checkins (energy, mood, missions_json, notes)

decisions (category, context_json, ai_response_json)

discipline_logs (log_type, reason)

weekly_reviews (ai_review_json)

5. Intent Extraction Architecture

All user messages are processed through deterministic LLM extraction.

Extraction schema must strictly conform to expected structure.

- intent
- extracted_data
- confidence

Invalid JSON must trigger repair attempt or fail safely.

6. Confirmation Workflow Design

High-risk updates require confirmation before persistence.

- Doctrine updates
- Daily check-in creation
- Reminder creation
- Discipline violations

Pending action stored with expiry time.

User confirmation resolves pending action.

7. Decision Engine Flow

Decision handler builds structured context including doctrine + daily state.

Calls OpenRouter model with strict JSON schema requirement.

Validates verdict schema before saving.

- verdict (approve|reject|delay)
- confidence (0–100)
- reasoning

- risks
- better_option
- next_steps

8. Reminder System Implementation

Time expressions extracted from chat.

Converted to UTC timestamp using Laravel Carbon.

Stored in reminders table.

Scheduler runs every minute to dispatch pending reminders.

Telegram message sent via Bot API.

9. End-of-Day Reflection Automation

Scheduled job runs daily at configured hour.

If check-in exists → send Telegram prompt.

User reply parsed and logged.

Optional stoic summary generated.

10. Telegram Webhook Implementation

Endpoint: POST /api/v1/telegram/webhook

Validate secret token header.

Extract telegram_chat_id.

Map to conversation record.

Forward message to ChatOrchestrator.

11. Web Chat Interface Design

Route: /chat

Uses same API endpoint /api/v1/chat

Displays conversation history.

Shows confirmation buttons for pending actions.

Uses subtle dark stoic aesthetic.

12. LLM Integration (OpenRouter)

Model: meta-llama/llama-3.3-70b-instruct

Temperature: 0.1–0.2

Max tokens: controlled to avoid runaway cost.

All prompts include JSON schema.

No markdown or prose allowed in JSON outputs.

13. Security Hardening

- Strict JSON validation
- Webhook signature validation

- Rate limiting chat endpoint
- Authorization enforcement
- Never trust raw LLM output

14. Scalability Considerations

Move LLM calls to queued jobs if latency increases.

Use Redis queue driver in production.

Implement conversation context trimming.

Add caching for dashboard summary endpoints.

15. Observability & Logging

Log all LLM raw responses (not user-visible).

Log JSON validation failures.

Attach request IDs for tracing.

Monitor reminder dispatch failures.

16. Frontend Integration Strategy

Auth persistence via localStorage token.

Axios interceptor attaches Bearer token.

Chat page streams responses if needed.

Command Center prefetches summary endpoints.

17. MVP Completion Checklist

- Chat via Telegram works
- Chat via Web works
- Doctrine via chat confirmed
- Check-in via chat confirmed
- Decision via chat returns verdict
- Reminders trigger
- End-of-day reflection works
- Dashboard shows structured history

18. Deployment Strategy

Use HTTPS for webhook endpoint.

Deploy Laravel on VPS or managed hosting.

Use Supervisor for queue workers.

Set up cron for Laravel scheduler.

19. Production Environment Variables

- APP_ENV=production
- OPENROUTER_API_KEY
- TELEGRAM_BOT_TOKEN
- TELEGRAM_SECRET_TOKEN
- QUEUE_CONNECTION=redis

20. Final Architectural Philosophy

Conversational outside, structured inside.

Strict JSON everywhere.

Confirmation before persistence.

Proactive discipline guidance.

Human-like but rule-bound.